

FLOWSTATE

AI-Powered Urban Flood Prediction and Safe Mobility Platform for Gurugram

PROJECT REPORT

Team Members: _____

Institution: _____

Course / Event: Software Engineering / Smart India Hackathon

Date: September 2026

1. Abstract

FlowState is a web-based urban flood intelligence platform designed for Gurugram. It combines weather forecasts, terrain and urban-surface characteristics, drainage condition and a road network to estimate waterlogging risk at street level. The platform provides a 0–3 hour forecast, early warnings, drainage status, an authority dashboard and flood-aware routing. The updated implementation uses Flask, SQLite/SQLAlchemy, React, Leaflet and a Random Forest model. Open-Meteo provides weather inputs; OpenStreetMap/Overpass can provide road geometry; Google Maps provides an optional navigation handoff.

2. Problem Statement

Urban flooding can rapidly make individual streets unsafe even when a city-wide warning is not sufficient for a driver or municipal operator. Decision-makers need localized information: where waterlogging is likely, how severe it may become, which drains are overloaded or blocked, when a road is expected to deteriorate and which alternative route avoids hazardous streets.

3. Objectives

- Predict street-level waterlogging risk for Gurugram.
- Provide 0–3 hour early warnings.
- Integrate rainfall forecasts and drainage condition.
- Visualize risk on an interactive map.
- Provide a shortest safe route that avoids dangerous road segments when possible.
- Give authorities an operational view of alerts, roads and drains.

4. Scope

The prototype focuses on the Gurugram urban area and is structured so that real road, drainage, DEM and historical flood datasets can replace the demonstration dataset without rewriting the prediction and visualization layers.

5. Technology Stack

Layer	Technology
Frontend	React 19, Vite, React-Leaflet, Leaflet
Backend	Python, Flask, Flask-CORS, SQLAlchemy
Database	SQLite
ML	scikit-learn Random Forest classifier + regressor
Weather	Open-Meteo API
Road data	OpenStreetMap / Overpass import
Routing	Local Dijkstra graph; OSRM documented as an optional routing service
Navigation	Google Maps Directions URL
Deployment	Gunicorn/Procfile for backend; Vite build for frontend

6. System Architecture

The platform is divided into data ingestion, prediction, drainage intelligence, routing, alerts, API and visualization layers. Weather data is cached by location. Road segments are joined to nearby Gurugram zones for terrain and drainage features. The prediction service evaluates every road independently. The map consumes these predictions and renders risk by line colour.

Architecture: Weather → Feature Engineering → ML/Physics → Street Risk → Map + Alerts + Routing. Drainage feeds the feature-engineering layer; the authority panel controls administrative operations.

7. Database Design

Table	Purpose
users	Authority/admin accounts and hashed passwords
zones	Gurugram spatial zones and physical features
drains	Drain location, capacity, blockage and status
route_segments	Road graph edges, geometry, road class and data source
flood_predictions	Optional persistence schema for road/horizon predictions
alerts	Active 3-hour road alerts

8. Prediction Methodology

The prediction layer uses rainfall history/forecast, elevation, slope, impervious surface, drainage utilization, blockage, historical flood frequency and road factor. The Random Forest classifier estimates the probability of waterlogging. A Random Forest regressor estimates water depth. Risk is then mapped to LOW, MODERATE, HIGH or SEVERE using probability/depth thresholds.

The current model is a synthetic calibration model created to make the application runnable and demonstrate the complete ML pipeline. It is explicitly not presented as validated Gurugram accuracy. A production model must be trained against observed flood labels.

9. 3-hour Early Warning

The system evaluates seven points: current, +30, +60, +90, +120, +150 and +180 minutes. Alert generation focuses on the +60, +120 and +180 minute windows and creates a warning when a road reaches HIGH or SEVERE risk. This makes the warning actionable rather than only reporting the current state.

10. Drainage Intelligence

Each drain has a nominal capacity and blockage percentage. Effective capacity is reduced by blockage. The prediction model uses aggregate drainage capacity and blockage to estimate utilization. A critical drain can therefore increase the risk score of nearby roads. The architecture is ready to replace simulated values with municipal asset data or IoT readings.

11. Safe Routing

Road segments are graph edges. Dijkstra is used to calculate the lowest travel-time path. In safe mode, HIGH and SEVERE segments are removed from the graph if possible. In balanced mode, risk is converted into a time penalty. This is materially different from adding a fixed detour distance and satisfies the requirement that the safe route should be the shortest available route through safe streets.

12. Map and User Experience

The dashboard shows the Gurugram street network with risk colours, drain markers, alerts and the selected route. A road click reveals its prediction and 3-hour timeline. The route panel lets a user compare normal, balanced and safe modes and continue navigation in Google Maps.

13. Administration

The authority panel uses a signed token generated after password verification. Protected actions include refreshing alerts and updating drain blockage. The design is intentionally lightweight for the hackathon. A production deployment should add refresh tokens, rate limiting, audit logs, CSRF protections where applicable, secret rotation and stronger identity management.

14. External API Integration

Open-Meteo is integrated for hourly precipitation and current weather. The service uses Asia/Kolkata and caches responses. Google Maps is integrated through a Directions URL handoff. OpenStreetMap/Overpass is supported through a backend import script. OSRM documentation is included as an optional routing-service path, but the updated safe-route logic is computed locally so flood filtering remains under FlowState control.

15. Testing and Verification

Check	Result
Python syntax compilation	Passed for updated backend source
Model training	Completed; metadata stored with model
Database design	Created through Flask CLI; seeded data included through seed service
Street prediction	Implemented as road-level endpoint and map layer
3-hour forecast	Implemented at 0/30/60/90/120/150/180 minutes
Safe routing	Dijkstra graph implementation included
Admin authentication	Signed token protection included
Open-Meteo fallback	Clearly labelled demo fallback included
Real OSM import	Script included; requires internet access
Frontend build	Requires npm install in the target environment; dependencies declared in package.json

16. Data and Scientific Limitations

The current demonstration model and infrastructure values are not sufficient for public-safety deployment. The project should not claim a real-world accuracy percentage based on synthetic data. The recommended next phase is a labelled Gurugram dataset built from historical rainfall, waterlogging observations, road geometry, drainage assets, elevation and land-use information. Validation should be time-based and storm-based, not only random train/test splitting.

17. Future Enhancements

- Import full OSM node/way topology for exact road graph routing.
- Use a high-resolution DEM and calculate slope/elevation per road segment.
- Connect verified municipal drainage assets and capacity data.
- Add rain gauges/radar/crowdsourced observations for real-time calibration.
- Add CCTV/image-based water-depth estimation as an optional computer-vision module.
- Persist every forecast run for backtesting and model monitoring.
- Add probabilistic calibration and uncertainty bands.
- Deploy scheduled ingestion and alert workers.

18. Conclusion

The updated FlowState architecture addresses the central requirements for a strong urban-flood hackathon prototype: street-level visualization, 3-hour prediction, drainage-aware risk, actionable alerts, authority controls and flood-aware safe routing. Its most important design decision is to keep data provenance visible: demo geometry and synthetic ML calibration are labelled as such, while Open-Meteo and OSM integrations are separated so they can be replaced by authoritative local datasets later.

19. References

Open-Meteo documentation — <https://open-meteo.com/en/docs>

Google Maps URLs — <https://developers.google.com/maps/documentation/urls/get-started>

OSRM HTTP API — <https://project-osrm.org/docs/v26.4.0/http>

OpenStreetMap Overpass API — https://wiki.openstreetmap.org/wiki/Overpass_API

Haryana / Gurugram topographic references used during review: Haryana State Pollution Control Board and related government planning documents. One HSPCB source reports Gurugram elevation at 224.6 m; another government planning document describes average district elevation around 217 m with a 190–280 m range.

Prepared as a technical project report for academic/hackathon use.